ECE 586

Computer Architecture
Final Project Report

HDL Implementation of a Global Branch Predictor with Indexing

Neeraj Mallanagouda Patil

CWID: A20582841

Abstract

This project uses and evaluates three dynamic global history branch predictors (gpredict, gselect and gshare) in Verilog HDL. The same Global History Register (4 bits) and 16-entry Branch History Table of 2-bit saturating counters are employed for each predictor, with the only difference in their design being how the BHT index is generated. All three are driven in parallel by a 6,000-branch trace from a nested-loop benchmark in a ModelSim test-bench. All three have 16.8% of mispredictions, and the more bits in the history the more the PC bits fail to distinguish the two branches, yet there is no way for indexing strategy to compensate for that except through increasing the length of the history.

1. Introduction

A modern pipelined processor which does not remember the outcome of the conditional branch will stall when the branch is unresolved at fetch. The branch prediction technique avoids this by speculatively fetching ahead of the branch and guessing the result, thus revealing more parallelism at the instruction level. The speculations are rewarded if correct and the pipeline must put down the speculations and re-run if incorrect, penalizing itself for mis-prediction. In a deeply pipelined design, it is therefore important to have high prediction accuracy.

This project is a Verilog implementation of three correlated dynamic predictors (all global history) and a comparison of their misprediction rate on a synthetic but representative branch trace. The first one is gpredict which indexes the Branch History Table (BHT) using only the Global History Register (GHR). The second gselect is a concatenation of the lower bits of the program counter with the lower bits of the GHR. The third gshare fetches the bit-wise exclusive-OR of the lower bits of the program counter and the GHR.

The main engineering challenge was the careful update protocol: predictions were read combinationally from the BHT, but both the indexed 2-bit saturating counter and the GHR had to be updated atomically (on a single clock edge) to ensure a consistent state for the next branch. The second challenge was to ensure that the three predictors could be fed from the same source and tested on the same conditions; this was addressed by instantiating all three predictors under a single testbench and by reading a binary trace file using $readmemb.

2. Background

2.1) Static and Dynamic Branch Prediction

Static prediction uses information in advance of run time. For instance, on the MIPS architecture all branches are assumed to be taken, on the Motorola 68000 all branches are assumed to be not taken [1]. Dynamic predictors watch outcomes as they are executed and adjust. The most straight-forward dynamic scheme is one-bit predictor that keeps track of the previous result of a branch, which fails twice per loop, once when entering and once when leaving the loop. Smith [5] introduced the 2-bit saturating counter, which introduces a hysteresis into the system so that a single counter-outcome does not flip the prediction, and is the counter used in this project.

2.2) The 2-bit Saturating Counter

Each BHT entry is for an 2-bit counter (as per project handout). If a outcome is taken, the counter is increased, up to 11, and if it is not taken, the counter is decreased, down to 00. The most-significant bit is the prediction, 1x is predict-taken and 0x is predict-not-taken.

2.3) Correlated Global Predictors

Single-counter (bimodal) predictors only consider a counter for a given branch and are unable to take advantage of correlations between successive branches. Yeh and Patt's scheme operates at two levels with a history register that indexes a table of counters [4]. However, by either concatenating the global history to the program counter (PC) or XORing them together (exclusive OR), they can obtain better accuracy with the same number of bits in the index when compared to either one alone [2]. All three of the architectures here discussed are chosen from McFarling's paper.

2.4) The Three Architectures Implemented

The n-bit GHR is the full address to a BHT of 2 to the n entries in gpredict. Each branch uses the same table, so each branch that has the same pattern of recent outcomes aliases to the same counter. In gselect, the m bits of the PC are combined with $(n - m)$ bits of the GHR to divide the BHT into per-PC regions, but retain some history per region. For gshare, the low n bits of the PC are XORed with the n-bit GHR meaning that the same (pc, history) pair is spread throughout the entire BHT, thereby reducing the aliasing for a given table size.

3. Architectural Exploration

3.1) Design Parameters

The parameters for all three predictors are the same as those called out in the project handout. The Global History Register is 4 bits wide and is set to 0000 (all not-taken). There are 16 entries in the Branch History Table (4-bit index), each entry is a 2-bit saturating counter that is initialized to 00 (strongly not-taken). The branch address is an 8-bit PC. The only thing that differs between the three modules is the index function, as summarized in Table 1.

Predictor	Index function
gpredict	index = ghr
gselect	index = { pc[1:0], ghr[1:0] }
gshare	index = pc[3:0] ^ ghr

Table 1: Index function for each predictor.

3.2) Hardware Cost

All three predictors are effectively the same in terms of their cost in hardware. They include 32 increment/decrement and saturating counters (16 two-bit each), 4 flip-flops (GHR), a 16 to 1 multiplexer to choose the prediction from the indexed counter, and a 4 to 16 one hot decoder, gated by the update signal. The differences are small: gpredict has no index logic, since it wires the index directly from the GHR, gselect only needs 4-bit concatenation (wiring without any logic gates) and gshare needs four 2-input XOR gates. The timing of the timing logic is only a few XOR gates and a few millimeters of routing away from the total area, so for any reasonable target technology, the three designs are area equivalent.

3.3) Trade-off Analysis on This Workload

The test program only has two distinct conditional branches: one at the inner loop back edge (PC 0x24) and one at the outer loop back edge (PC 0x30). This benchmark sheds light on two key and learning points of the selected index functions.

Support for PC bit aliasing in gselect. In binary, 0x24 = 00100100 and 0x30 = 00110000. The low two bits of both PCs are identical (00). Since the contributions of the PC to gselect's index are identical in both branches, the index simplifies to {00, ghr[1:0]} for both branches, for whichever branch is being predicted. Here, four BHT entries are accessed in this workload, and the two branches alias completely, so there is no benefit for 4-bit gselect over gpredict.

Insufficient history length. The inner loop has period 5: 4 taken and 1 not-taken. The steady state composite branch sequence is 6 elements, T T T T NT T (5 inner branches, followed by one outer branch). After warm-up, the GHR cycles through the six values 1101, 1011, 0111, 1111, 1111, and 1110. The number 1111 is repeated in this cycle, at the fourth inner branch (where the true output occurs) and at the fifth inner branch (where the true output does not occur). These two cases are indistinguishable with 4 bits of history as the predictor can't tell them apart, and any index function using only (PC, 4-bit GHR) will result in contradictory output at the same BHT entry and oscillate.

This is why gshare, which XOR-mixes PC with history, with this trace is no better than gpredict: it is a choice between 2 visits to the same branch at the same GHR, not a choice between 2 branches. XOR alters the entire BHT mapping for that branch but is unable to create anything that the GHR couldn’t store.

4. Functional Validation and Verification

4.1) Test Program and Trace Generation

The nested-loop benchmark (from handout) is used for the test program. It was hand-translated to MIPS assembly (Appendix A): the inner loop branch is at byte offset 0x24 and the outer loop branch is at 0x30. The dynamic outcome stream was generated directly with a Python script (Appendix B) emulating the loop semantics, instead of running the assembly on a simulated MIPS pipeline. The script creates two files. The first, labeled trace.txt, is a human readable listing where each line is a line of a branch in the form 0xPP T or 0xPP NT. The second, called "trace.mem", is a binary representation per line of ModelSim's $readmemb directive, with bits [8:1] representing the PC and bit [0] representing the taken outcome.

The script produced exactly 6,000 branches: 5,000 at PC 0x24 (4,000 taken plus 1,000 not-taken) and 1,000 at PC 0x30 (999 taken plus 1 not-taken). This is consistent with the project handout estimate of ~5000 branch decisions.

4.2) Testbench Architecture

All three predictors are instantiated in a single Verilog testbench (Appendix G) using the same clock, reset, PC-bus, actual-outcome-bus, and one-cycle update-pulse. At the start of simulation, the trace is loaded in a 9-bit-wide memory using $readmemb. The predictor inputs are loaded with the PC and the actual output for each branch. Following a short settle delay the combinational predict_taken outputs of all three predictors are sampled, and on a mismatch the appropriate miss counter is incremented. The update

signal is then pulsed for one clock cycle, at the rising edge of the pulse, the indexed BHT counter increments or decrements, with saturation, and the GHR shifts in the actual result.

All three predictors are driven from the same testbench at the same time, so that any misprediction count difference is due to the index function only.

4.3) Simulation Setup

The simulation was done using Mentor Graphics ModelSim DE 2025.2 in the ECE department's Rocky Linux server on the IIT VPN. The ModelSim do-file in Appendix H automates the compile, elaborate and run sequence. All 6,000 branches were processed by the simulator and $finish was reached at simulated time 60,015 ns.

4.4) Waveform Inspection

Three waveform views from ModelSim were captured to verify correct functional behavior.

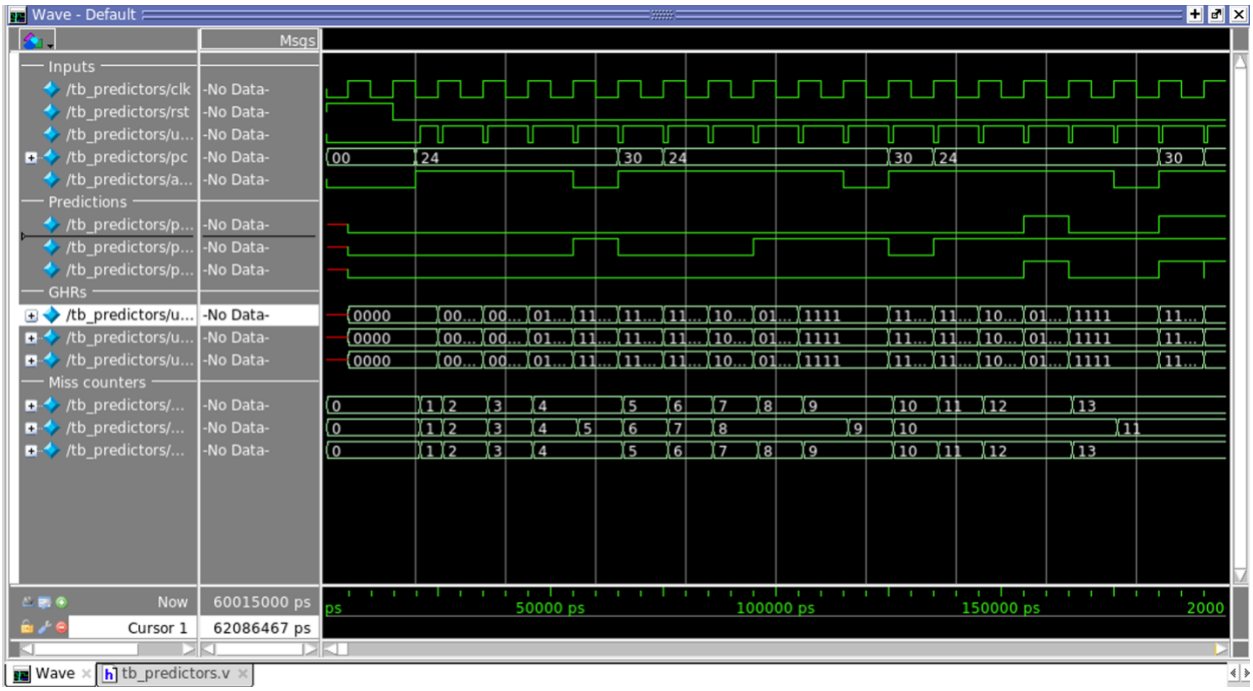

Figure 1. Early simulation: reset, GHR initialization, warm-up.

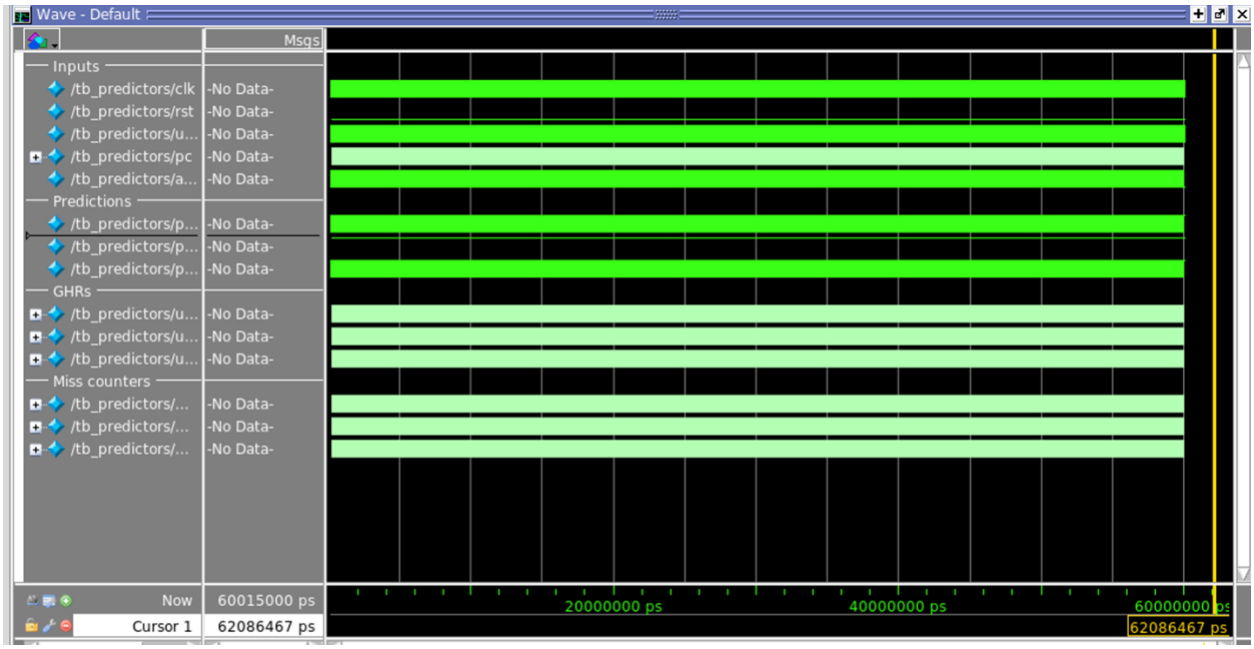

Figure 2. Full simulation at zoom-full.

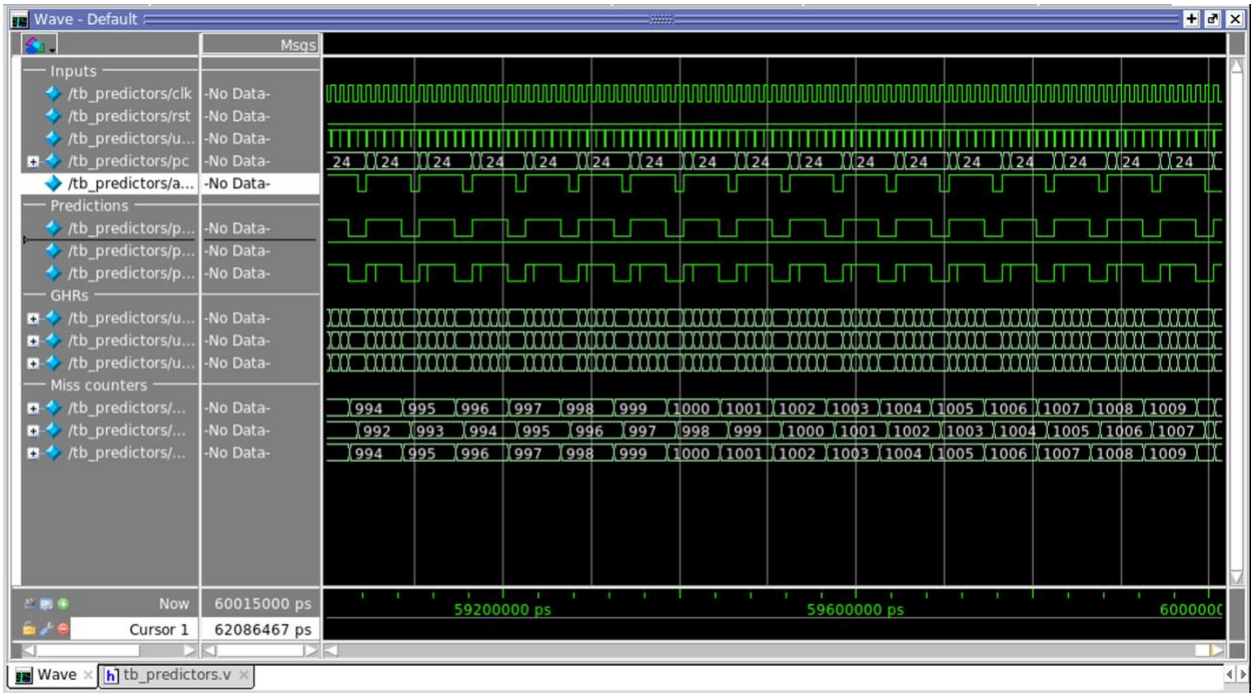

Figure 3. Steady-state behavior approaching simulation end.

The first two clock cycles are verified by resetting behavior in which all three GHRs are initialized to 0000, and all miss counters are initialized to 0. GHR shifting was verified against the expected pattern 0000, 0001, 0011, 0111, 1111, 1110 for the first six branches. To test for steady state behavior, it was observed that the miss counts of all three predictors increase at the same rate after warming up, as per the analysis.

5. Results

5.1) Misprediction Counts

```
#
# ================================================================
#                  Branch Prediction Results
# ================================================================
# Total branches simulated: 6000
#
# Predictor    Mispredicts    Mispredict Rate    Hit Rate
# ---------    -----------    ---------------    --------
# gpredict        1011          16.85%            83.15%
# gselect         1009          16.82%            83.18%
# gshare          1011          16.85%            83.15%
# ================================================================
# ** Note: $finish    : ../hdl/tb_predictors.v(111)
#    Time: 60015 ns  Iteration: 1  Instance: /tb_predictors
# 1
# Break in Module tb_predictors at ../hdl/tb_predictors.v line 111
# 0 ps
# 63015750 ps
```

Table 2 summarizes the misprediction counts and rates produced by the simulation.

Predictor	Mispredictions	Miss Rate	Hit Rate
gpredict	1011	16.85%	83.15%
gselect	1009	16.82%	83.18%
gshare	1011	16.85%	83.15%

Table 2. Misprediction counts and rates over 6,000 branches.

5.2) Discussion

The three predictors are all within 2 mispredictions of each other. It is not a bug, it is the expected behavior of the parameters and workload, and it shows two general principles about correlated branch prediction.

Most of the mispredictions are generated by a single, structurally irregular branch. About 1000 of the 1011 misses are on the inner loop exit (5th inner loop branch of each outer loop). There is a GHR of 1111 at that branch and at the fourth inner branch, which is taken, there is also a GHR of 1111. The two cases can't be distinguished with just 4 bits of history. An LFSR with two bits is saturated at the corresponding BHT entry, oscillating and mis predicting one misprediction per outer iteration. The rest of the few mispredictions are distributed throughout BHT warm up at the beginning of the simulation, with all counters at 00 (strongly not taken), but most actual outcomes taken.

For gselect, the ambiguity of inner-vs-outer can't be resolved by indexing strategy since the bits of PC used for both branches are the same; and for gshare, the ambiguity is between two visits to the same branch at the same GHR, so XOR-mixing will yield the same index in either case. Gselect's tiny misprediction advantage is purely due to its chance to take a marginally better route through the BHT during warmup, in the lead up to steady state.

5.3) Implications for Predictor Design

To minimize these mispredictions on this workload the GHR needs to be at least 5 bits wide, which allows the inner-loop duration of 5 to fit into the history. The fourth and fifth inner branches have a 5-bit history, and they differ in their fifth-most-recent outcome, which the predictor can disambiguate. It agrees with McFarling's overall findings that the benefit of gshare over gpredict increases as the table size and history length increase [2]. This is a worst-case scenario for differentiating between the three indexing schemes, since it has only 4 bits of history, and an aliased PC pair.

6. Conclusion and Future Work

A doubly-nested loop was used to generate a global-history branch predictor trace of length 6,000, which was implemented in three different branch predictors — gpredict, gselect and gshare — in Verilog and compared. It is possible to compare the two implementations directly, with control, since they share a common interface with only one difference – the BHT index function. They both had a misprediction rate of about 16.8%, and, after careful examination, it turned out that those two factors of the experimental setup are causing them to be near equal: the period of the inner loop (5) is larger than the GHR (4 bits) and the low 2 bits of the PC pair are the same, so gselect has no PC information to use.

There would be some extensions that would expand on this work. It seems obvious to start by increasing the GHR to 5 or 8 bits, and the BHT accordingly, in order to see the expected benefits of gshare over gpredict with capturing the full inner-loop period. If the selection of PC field were increased, such as the low 4 bits, then the two branches would go to different BHT regions, and the aliasing would be non-degenerate here. A further extension is a tournament predictor which adaptively switches between the gshare and a simple bimodal predictor on a per-branch basis, as suggested by McFarling. The predictors would expose realistic aliasing behavior if they were run on a

richer trace (one with more than two unique branches) such as an instruction-level trace of a SPECint kernel. Finally, the design can be synthesized to an FPGA target for area and worst-case path delay measurement for each predictor.

Appendix: Source Code

A. branch_test.s: MIPS Assembly

```
.data
c:      .word   0
        .text
        .globl  main
main:
        la      $s0, c
        addi    $t0, $zero, 0
outer_loop:
        addi    $t1, $zero, 0
inner_loop:
        lw      $t4, 0($s0)
        addi    $t4, $t4, 1
        sw      $t4, 0($s0)
        addi    $t1, $t1, 1
        slti    $t3, $t1, 5
        bne     $t3, $zero, inner_loop
        addi    $t0, $t0, 1
        slti    $t2, $t0, 1000
        bne     $t2, $zero, outer_loop
end:
        lw      $v0, 0($s0)
        jr      $ra
```

B. gen_trace.py: Python Trace Generator

```
INNER_PC = 0x24
OUTER_PC = 0x30
def generate_trace():
    trace = []
    for i in range(1000):
        for j in range(5):
            taken = (j + 1) < 5
            trace.append((INNER_PC, taken))
        taken = (i + 1) < 1000
        trace.append((OUTER_PC, taken))
    return trace
def main():
    trace = generate_trace()
    with open("trace.txt", "w") as f:
        for pc, taken in trace:
            f.write(f"0x{pc:02X} {'T' if taken else 'NT'}\n")
    with open("trace.mem", "w") as f:
        for pc, taken in trace:
            f.write(f"{pc:08b}{1 if taken else 0}\n")
    total = len(trace)
    takens = sum(1 for _, t in trace if t)
    inner = sum(1 for pc, _ in trace if pc == INNER_PC)
    outer = sum(1 for pc, _ in trace if pc == OUTER_PC)
    print(f"Generated {total} branches")
```

```
    print(f"  Taken     : {takens}")
    print(f"  Not taken : {total - takens}")
    print(f"  Inner @ 0x{INNER_PC:02X} : {inner}")
    print(f"  Outer @ 0x{OUTER_PC:02X} : {outer}")
    print()
    print("Wrote: trace.txt, trace.mem")
if __name__ == "__main__":
    main()
```

C. sat_counter.v: 2-bit Saturating Counter

```
module sat_counter (
    input  wire clk,
    input  wire rst,
    input  wire enable,
    input  wire taken,
    output wire predict_taken
);
    reg [1:0] count;
    always @(posedge clk) begin
        if (rst)
            count <= 2'b00;
        else if (enable) begin
            if (taken && count != 2'b11)
                count <= count + 1'b1;
            else if (!taken && count != 2'b00)
                count <= count - 1'b1;
        end
    end
    assign predict_taken = count[1];
endmodule
```

D. gpredict.v: Global Predictor

```
module gpredict (
    input  wire       clk,
    input  wire       rst,
    input  wire       update,
    input  wire [7:0] pc,
    input  wire       actual_taken,
    output wire       predict_taken
);
    reg [3:0] ghr;
    wire [3:0] index;
    wire [15:0] counter_predict;
    wire [15:0] counter_enable;
    assign index = ghr;
    assign counter_enable = update ? (16'h0001 << index) : 16'h0000;
    genvar k;
    generate
        for (k = 0; k < 16; k = k + 1) begin : bht
            sat_counter sc (
                .clk           (clk),
                .rst           (rst),
                .enable        (counter_enable[k]),
                .taken         (actual_taken),
                .predict_taken (counter_predict[k])
```

```
            );
        end
    endgenerate
    assign predict_taken = counter_predict[index];
    always @(posedge clk) begin
        if (rst)
            ghr <= 4'b0000;
        else if (update)
            ghr <= {ghr[2:0], actual_taken};
    end
endmodule
```

E. gselect.v: Global Predictor with Index Selection

```
module gselect (
    input  wire       clk,
    input  wire       rst,
    input  wire       update,
    input  wire [7:0] pc,
    input  wire       actual_taken,
    output wire       predict_taken
);
    reg  [3:0]  ghr;
    wire [3:0]  index;
    wire [15:0] counter_predict;
    wire [15:0] counter_enable;
    assign index = { pc[1:0], ghr[1:0] };
    assign counter_enable = update ? (16'h0001 << index) : 16'h0000;
    genvar k;
    generate
        for (k = 0; k < 16; k = k + 1) begin : bht
            sat_counter sc (
                .clk           (clk),
                .rst           (rst),
                .enable        (counter_enable[k]),
                .taken         (actual_taken),
                .predict_taken (counter_predict[k])
            );
        end
    endgenerate
    assign predict_taken = counter_predict[index];
    always @(posedge clk) begin
        if (rst)
            ghr <= 4'b0000;
        else if (update)
            ghr <= {ghr[2:0], actual_taken};
    end
endmodule
```

F. gshare.v: Global Predictor with Index Sharing

```
module gshare (
    input  wire       clk,
    input  wire       rst,
    input  wire       update,
```

```
    input  wire [7:0] pc,
    input  wire       actual_taken,
    output wire       predict_taken
);

    reg  [3:0]  ghr;
    wire [3:0]  index;
    wire [15:0] counter_predict;
    wire [15:0] counter_enable;
    assign index = pc[3:0] ^ ghr;
    assign counter_enable = update ? (16'h0001 << index) : 16'h0000;
    genvar k;
    generate
        for (k = 0; k < 16; k = k + 1) begin : bht
            sat_counter sc (
                .clk           (clk),
                .rst           (rst),
                .enable        (counter_enable[k]),
                .taken         (actual_taken),
                .predict_taken (counter_predict[k])
            );
        end
    endgenerate
    assign predict_taken = counter_predict[index];
    always @(posedge clk) begin
        if (rst)
            ghr <= 4'b0000;
        else if (update)
            ghr <= {ghr[2:0], actual_taken};
    end
endmodule
```

G. tb_predictors.v: Verilog Testbench

```
`timescale 1ns/1ps
module tb_predictors;
    reg        clk;
    reg        rst;
    reg        update;
    reg  [7:0] pc;
    reg        actual_taken;
    wire pred_g, pred_gs, pred_gh;
    gpredict u_g  (.clk(clk), .rst(rst), .update(update),
                   .pc(pc), .actual_taken(actual_taken),
                   .predict_taken(pred_g));
    gselect  u_gs (.clk(clk), .rst(rst), .update(update),
                   .pc(pc), .actual_taken(actual_taken),
                   .predict_taken(pred_gs));
    gshare   u_gh (.clk(clk), .rst(rst), .update(update),
                   .pc(pc), .actual_taken(actual_taken),
                   .predict_taken(pred_gh));
    always #5 clk = ~clk;
    reg [8:0] trace [0:7999];
    integer   N;
    integer   miss_g, miss_gs, miss_gh;
    integer   i;
```

```
    reg [7:0] entry_pc;
    reg       entry_taken;
    initial begin
        $readmemb("trace.mem", trace);
        N = 0;
        while (N < 8000 && trace[N] !== 9'bx) N = N + 1;
        $display("[TB] Loaded %0d branches from trace.mem", N);
        clk = 0; rst = 1; update = 0; pc = 8'h00; actual_taken = 1'b0;
        miss_g = 0; miss_gs = 0; miss_gh = 0;
        @(posedge clk); @(posedge clk);
        rst = 1'b0;
        @(negedge clk);
        for (i = 0; i < N; i = i + 1) begin
            entry_pc    = trace[i][8:1];
            entry_taken = trace[i][0];
            pc = entry_pc; actual_taken = entry_taken; update = 1'b0;
            #1;
            if (pred_g  !== entry_taken) miss_g  = miss_g  + 1;
            if (pred_gs !== entry_taken) miss_gs = miss_gs + 1;
            if (pred_gh !== entry_taken) miss_gh = miss_gh + 1;
            update = 1'b1;
            @(posedge clk);
            update = 1'b0;
        end
        $display("");
        $display("========================================");
        $display("        Branch Prediction Results");
        $display("========================================");
        $display("Total branches simulated: %0d", N);
        $display("Predictor    Mispredicts    Miss Rate    Hit Rate");
        $display("gpredict     %5d          %6.2f%%      %6.2f%%",
                 miss_g,  100.0*miss_g/N,  100.0*(N-miss_g)/N);
        $display("gselect      %5d          %6.2f%%      %6.2f%%",
                 miss_gs, 100.0*miss_gs/N, 100.0*(N-miss_gs)/N);
        $display("gshare       %5d          %6.2f%%      %6.2f%%",
                 miss_gh, 100.0*miss_gh/N, 100.0*(N-miss_gh)/N);
        $finish;
    end
endmodule
```

H. run.do: ModelSim Simulation Script

```
if {![file isdirectory work]} { vlib work }
vlog ../hdl/sat_counter.v
vlog ../hdl/gpredict.v
vlog ../hdl/gselect.v
vlog ../hdl/gshare.v
vlog ../hdl/tb_predictors.v
vsim work.tb_predictors
add wave -divider "Inputs"
add wave -radix bin sim:/tb_predictors/clk
add wave -radix bin sim:/tb_predictors/rst
add wave -radix bin sim:/tb_predictors/update
add wave -radix hex sim:/tb_predictors/pc
add wave -radix bin sim:/tb_predictors/actual_taken
add wave -divider "Predictions"
```

```
add wave -radix bin sim:/tb_predictors/pred_g
add wave -radix bin sim:/tb_predictors/pred_gs
add wave -radix bin sim:/tb_predictors/pred_gh
add wave -divider "GHRs"
add wave -radix bin sim:/tb_predictors/u_g/ghr
add wave -radix bin sim:/tb_predictors/u_gs/ghr
add wave -radix bin sim:/tb_predictors/u_gh/ghr
add wave -divider "Miss counters"
add wave -radix unsigned sim:/tb_predictors/miss_g
add wave -radix unsigned sim:/tb_predictors/miss_gs
add wave -radix unsigned sim:/tb_predictors/miss_gh
run -all
wave zoom full
```

References

[1] ECE 586 Final Project, Spring 2026: HDL Implementation of a Global Predictor with Indexing. Project handout, Illinois Institute of Technology, April 2026.

[2] S. McFarling. Combining Branch Predictors. WRL Technical Note TN-36, Digital Equipment Corporation Western Research Laboratory, June 1993.

[3] J. L. Hennessy and D. A. Patterson. Computer Architecture: A Quantitative Approach, 6th ed. Morgan Kaufmann, 2017. Chapter 3, Branch Prediction.

[4] T.-Y. Yeh and Y. N. Patt. Two-level adaptive training branch prediction. In Proceedings of the 24th Annual International Symposium on Microarchitecture (MICRO-24), 1991, pp. 51-61.

[5] J. E. Smith. A Study of Branch Prediction Strategies. In Proceedings of the 8th Annual International Symposium on Computer Architecture (ISCA), 1981, pp. 135-148.

[6] Computer Architecture Visualization: gshare Branch Prediction. http://comparchviz.com.s3-website-us-east-1.amazonaws.com/gshare.html (linked in project handout, accessed May 2026).